ALGOTRADING SYSTEM — ENHANCED MASTER REQUIREMENTS V3

O(1) Guaranteed | QuestDB | Full Market Depth | Top Gainers/Losers | PRECISE DHAN API MAPPINGS

All Timeframes: 5s, 10s, 15s, 20s, 25s, 30s, 35s, 40s, 45s, 50s, 55s, 60s + 1m to 30m **Stack:** Rust + Tokio + QuestDB + Redis | Docker-First | AWS + Local Parity **Document Date:** February 2026 | **Status:** READY FOR IMPLEMENTATION **PRECISE REQUIREMENTS FOR CLAUDE CODE IMPLEMENTATION**

TABLE OF CONTENTS

1. Critical Changes from V2 to V3
2. QuestDB: The O(1) Database
3. Expanded Candle Timeframes
4. Top Gainers & Top Losers
5. Dhan Live Market Feed — PRECISE Binary Protocol Specification
6. Dhan Full Market Depth — 20-Level & 200-Level PRECISE Specification
7. Dhan Annexure — Exchange Segments, Instruments, Feed Codes, Error Codes
8. Dhan Authentication & Token Management
9. Dhan Order Management API — PRECISE Specification
10. Dhan Live Order Update WebSocket
11. O(1) Component Scoring
12. Complete Rust Crate Dependency Map
13. Docker-First Stack
14. Latency Budget
15. End-to-End O(1) Processing Pipeline
16. Warmup Data & Historical Data
17. Deliverables Checklist
18. Final Summary

---

1. CRITICAL CHANGES FROM V2 TO V3

This document **supersedes** V2 with these critical additions:

Change	From (V2)	To (V3)	Impact
Dhan API Mapping	Generic references	Precise byte-level binary protocol specs	Exact Rust parser implementation
Live Feed Header	Assumed 12-byte	Actual 8-byte header (per Dhan docs)	Correct binary parsing
Feed Request Codes	Only code 23 mentioned	All codes: 11,12,15-18,21-24 mapped	Complete subscription logic
Feed Response Codes	Partial	Full: 1,2,4,5,6,7,8,50 mapped	Complete response routing
Exchange Segments	Named only	Precise enum values (0-8) mapped	Correct binary segment parsing
Depth Packet Format	Assumed 16-byte	Actual per-level structure documented	Correct depth parsing
200-Level Depth	Single instrument noted	Full subscription/response structure	Complete 200-depth integration
Order API	Generic POST	Precise endpoints, headers, JSON body	Working order placement
Authentication	TOTP mentioned	Full token generation + renewal flow	Automated 24-hour token cycle
Static IP	Mentioned	Full API for set/get/modify IP	SEBI compliance automation
Connection Limits	Approximate	Precise: 5 WS + 50 instruments/depth + 1/200-depth	Correct connection planning
Disconnection	Not specified	Binary disconnect packet + error codes	Graceful reconnection
Endianness	Not specified	Little Endian confirmed	Correct byte order parsing
Instrument Types	Partial list	Full: INDEX, FUTIDX, OPTIDX, EQUITY, FUTSTK, OPTSTK, FUTCOM, OPTFUT, FUTCUR, OPTCUR	Complete instrument handling

2. QuestDB: THE O(1) DATABASE

2.1 Why QuestDB Achieves O(1)

Capability	TimescaleDB	QuestDB	Winner
Write (append)	O(1) batched	O(1) append-only columnar	Tie
Read individual tick	O(log N) B-tree	O(1) via partition pruning + materialized view	QuestDB
OHLCV candle query	O(N) aggregation	O(1) from materialized view (pre-computed)	QuestDB
Top gainers/losers	O(N) full scan + sort	O(1) from LATEST ON + materialized view	QuestDB
Ingestion throughput	~500K rows/sec	Millions of rows/sec (ILP protocol)	QuestDB
Timestamp precision	Microsecond	Nanosecond (TIMESTAMP_NS)	QuestDB
Array columns	Not supported	N-dimensional arrays (v9.0+)	QuestDB
Rust client	tokio-postgres (generic)	questdb-rs (native ILP) + tokio-postgres (reads)	QuestDB
Materialized views	Manual refresh	REFRESH IMMEDIATE (auto-incremental)	QuestDB

2.2 QuestDB Architecture — Three-Tier Storage

Tier 1 — WAL: Incoming writes → WAL → durability. Buffers out-of-order data. Millions of rows/sec. **Tier 2 — Columnar Partitions:** Time-partitioned columnar files. SIMD-accelerated scans. **Tier 3 — Parquet/Object Storage:** Aged data → object storage automatically. 90%+ cost reduction.

2.3 QuestDB Rust Integration

Ingestion (questdb-rs via ILP):

- Crate: `questdb-rs` (latest, with `ndarray` feature for 2D order book arrays)
- Protocol: ILP over HTTP (recommended) or TCP
- Async: Full Tokio async/await compatible
- Arrays: Native Rust slices, `Vec<Vec<f64>>`, ndarray for order book depth

Query (tokio-postgres via PGWire):

- Crate: `tokio-postgres` 0.7.x with `chrono` feature
- Connection: `host=localhost port=8812 user=admin password=quest dbname=qdb`
- Timestamps: `DateTime<Utc>` or `NaiveDateTime` via chrono

2.4 QuestDB Schema Design

Base Tables:

```sql
```

```sql
-- Raw ticks from Dhan Live Market Feed
CREATE TABLE ticks (
    symbol SYMBOL,
    security_id INT,
    exchange_segment INT,       -- Dhan enum: 0=IDX_I, 1=NSE_EQ, 2=NSE_FNO, etc.
    ltp DOUBLE,
    ltq INT,                    -- Last Traded Quantity (from Quote/Full packet)
    ltt INT,                    -- Last Trade Time EPOCH (from Ticker/Quote/Full)
    atp DOUBLE,                 -- Average Trade Price (from Quote/Full)
    volume INT,
    total_sell_qty INT,         -- From Quote/Full packet
    total_buy_qty INT,          -- From Quote/Full packet
    oi INT,                     -- Open Interest (from OI packet or Full packet)
    day_open DOUBLE,            -- From Quote/Full packet
    day_close DOUBLE,           -- From Quote/Full packet (post-market only)
    day_high DOUBLE,            -- From Quote/Full packet
    day_low DOUBLE,             -- From Quote/Full packet
    prev_close DOUBLE,          -- From Prev Close packet (response code 6)
    prev_oi INT,                -- Previous day OI (from Prev Close packet)
    change_pct DOUBLE,          -- Computed: (ltp - prev_close) / prev_close * 100
    highest_oi INT,             -- Highest OI for the day (NSE_FNO only, from Full)
    lowest_oi INT,              -- Lowest OI for the day (NSE_FNO only, from Full)
    timestamp TIMESTAMP
) TIMESTAMP(timestamp) PARTITION BY HOUR;

-- 20-Level Market Depth snapshots
CREATE TABLE depth_20 (
    symbol SYMBOL,
    security_id INT,
    exchange_segment INT,
    bids DOUBLE[][],            -- [[price, qty, orders], ...] x 20 levels
    asks DOUBLE[][],            -- [[price, qty, orders], ...] x 20 levels
    timestamp TIMESTAMP
) TIMESTAMP(timestamp) PARTITION BY HOUR;

-- 200-Level Market Depth snapshots
CREATE TABLE depth_200 (
    symbol SYMBOL,
    security_id INT,
    exchange_segment INT,
    num_bid_rows INT,           -- From 200-level header: actual number of rows
    num_ask_rows INT,
    bids DOUBLE[][],            -- [[price, qty, orders], ...] x up to 200 levels
    asks DOUBLE[][],            -- [[price, qty, orders], ...] x up to 200 levels
    timestamp TIMESTAMP
) TIMESTAMP(timestamp) PARTITION BY HOUR;
```

```sql
-- Orders placed/modified/cancelled via Dhan API
CREATE TABLE orders (
    order_id SYMBOL,            -- Dhan orderId string
    correlation_id SYMBOL,      -- Our internal correlation ID
    symbol SYMBOL,
    security_id INT,
    exchange_segment SYMBOL,    -- "NSE_EQ", "NSE_FNO", etc.
    transaction_type SYMBOL,    -- "BUY" or "SELL"
    product_type SYMBOL,        -- "CNC", "INTRADAY", "MARGIN", "CO", "BO"
    order_type SYMBOL,          -- "MARKET", "LIMIT", "SL", "SLM"
    validity SYMBOL,            -- "DAY", "IOC"
    quantity INT,
    price DOUBLE,
    trigger_price DOUBLE,
    traded_qty INT,
    traded_price DOUBLE,
    status SYMBOL,              -- "TRANSIT","PENDING","REJECTED","CANCELLED","PART_TRADED","TRADED"
    side SYMBOL,
    timestamp TIMESTAMP
) TIMESTAMP(timestamp) PARTITION BY DAY;
```

2.5 Materialized Views for O(1) Reads

Every candle timeframe gets its own materialized view with REFRESH IMMEDIATE:

```sql
```

```sql
-- 5-second candles
CREATE MATERIALIZED VIEW candles_5s REFRESH IMMEDIATE AS
SELECT timestamp, symbol,
   first(ltp) AS open, max(ltp) AS high, min(ltp) AS low, last(ltp) AS close,
   sum(volume) AS volume, last(oi) AS oi
FROM ticks SAMPLE BY 5s;

-- 10-second candles
CREATE MATERIALIZED VIEW candles_10s REFRESH IMMEDIATE AS
SELECT timestamp, symbol,
   first(ltp) AS open, max(ltp) AS high, min(ltp) AS low, last(ltp) AS close,
   sum(volume) AS volume, last(oi) AS oi
FROM ticks SAMPLE BY 10s;

-- ... repeat for: 15s, 20s, 25s, 30s, 35s, 40s, 45s, 50s, 55s, 60s
-- ... and for: 1m, 2m, 3m, 4m, 5m, 10m, 15m, 25m, 30m

-- Latest prices for top gainers/losers scanner
CREATE MATERIALIZED VIEW latest_prices REFRESH IMMEDIATE AS
SELECT timestamp, symbol,
   last(ltp) AS ltp, last(prev_close) AS prev_close,
   last(change_pct) AS change_pct, last(volume) AS volume, last(oi) AS oi
FROM ticks SAMPLE BY 1s;
```

Total materialized views: 21 (candle timeframes) + 1 (latest_prices) = 22

---

3. EXPANDED CANDLE TIMEFRAMES

3.1 Complete Timeframe Matrix (21 Total)

Category	Timeframes	Count	Implementation
Sub-minute (5s increments)	5s, 10s, 15s, 20s, 25s, 30s, 35s, 40s, 45s, 50s, 55s, 60s	12	QuestDB SAMPLE BY Ns
Standard minutes	1m, 2m, 3m, 4m, 5m	5	QuestDB SAMPLE BY Nm
Extended minutes	10m, 15m, 25m, 30m	4	QuestDB SAMPLE BY Nm
TOTAL		**21**	Each with own materialized view

3.2 Candle Generation: O(1) Per Tick Per Timeframe

Per candle state (in-memory):

- **Open:** First tick price in period (set once, O(1))
- **High:** `max(current_high, tick_price)` — single comparison, O(1)
- **Low:** `min(current_low, tick_price)` — single comparison, O(1)
- **Close:** Latest tick price (overwrite, O(1))
- **Volume:** Cumulative sum (addition, O(1))
- **OI:** Latest open interest (overwrite, O(1))

Period boundary detection: `timestamp % period_seconds == 0`. Pure arithmetic, O(1). At boundary: flush completed candle to QuestDB via ILP, reset state for new period.

---

4. TOP GAINERS & TOP LOSERS

4.1 Architecture

In-Memory State: `HashMap<SecurityId, PriceState>` with 214 entries (205 F&O stocks + 9 indices).

On each tick:

1. Update price state — O(1) HashMap lookup + O(1) update
2. Compute `change_pct = (ltp - prev_close) / prev_close * 100` — O(1) arithmetic
3. Update sorted position in `BTreeMap<OrderedFloat<f64>, SecurityId>` — O(log 214) ≈ 7.7 = effectively O(1)

4.2 Categories Tracked

Category	Metric	Universe	Update Frequency
Top Price Gainers	% change from prev close (positive)	All 214 F&O	Every tick
Top Price Losers	% change from prev close (negative)	All 214 F&O	Every tick
Top Volume Gainers	Absolute volume increase	All 214 F&O	Every tick
Top OI Gainers	OI increase from prev day	All 214 F&O	Every tick
Top OI Losers	OI decrease from prev day	All 214 F&O	Every tick
Most Active by Value	LTP × Volume (turnover)	All 214 F&O	Every tick

4.3 Filters

Filter	Count	Query Pattern
All F&O Underlyings	214	No filter
F&O Stocks Only	205	`WHERE instrument_type IN ('FUTSTK','OPTSTK')`
F&O Indices Only	9	`WHERE instrument_type IN ('FUTIDX','OPTIDX')`
By Sector	Variable	`WHERE sector = 'Banking'`
By Exchange	NSE/BSE	`WHERE exchange = 'NSE'`

---

5. DHAN LIVE MARKET FEED — PRECISE BINARY PROTOCOL SPECIFICATION

> **Source:** https://dhanhq.co/docs/v2/live-market-feed/

5.1 WebSocket Connection

Endpoint:

```
wss://api-feed.dhan.co?version=2&token={ACCESS_TOKEN}&clientId={CLIENT_ID}&authType=2
```

Query Parameters:

Field	Required	Value	Description
`version`	YES	`2`	DhanHQ v2
`token`	YES	JWT string	Access Token generated via Dhan
`clientId`	YES	String	User-specific identification generated by Dhan
`authType`	YES	`2`	Default value

Connection Limits:

- Maximum **5 WebSocket connections** per user
- Maximum **5000 instruments** per connection
- Maximum **100 instruments** per single JSON subscription message (send multiple messages to subscribe all)
- If >5 WebSocket connections established, the **first socket disconnects** with error code `805`

5.2 Subscribing to Instruments

JSON Request Structure (sent over WebSocket):

```json
{
  "RequestCode": 15,
  "InstrumentCount": 2,
  "InstrumentList": [
    {
      "ExchangeSegment": "NSE_EQ",
      "SecurityId": "1333"
    },
    {
      "ExchangeSegment": "NSE_FNO",
      "SecurityId": "49081"
    }
  ]
}
```

RequestCode values for subscription:

RequestCode	Action
15	Subscribe — Ticker Packet
16	Unsubscribe — Ticker Packet
17	Subscribe — Quote Packet
18	Unsubscribe — Quote Packet
21	Subscribe — Full Packet (LTP + Quote + OI + 5-Level Depth)
22	Unsubscribe — Full Packet

IMPLEMENTATION NOTE: For our algotrading system, use **RequestCode 21 (Full Packet)** for all 214 F&O underlyings. This gives us LTP, Quote, OI, and 5-level market depth in a single packet — the most comprehensive data mode.

5.3 Keeping Connection Alive

- Server sends **ping every 10 seconds**
- WebSocket library must auto-respond with **pong**
- If client does not respond for **>40 seconds**, connection is **closed by server**

- **Implementation:** Tokio-tungstenite handles ping-pong automatically. Add watchdog timer to detect stale connections and trigger reconnection.

5.4 Binary Response Format — CRITICAL DETAILS

Endianness: ALL DATA IS LITTLE ENDIAN

If the system is Big Endian, byte order conversion is required. Rust on x86/ARM is Little Endian natively, so no conversion needed.

5.4.1 Response Header (8 bytes — Common to ALL packets)

Byte Position	Type	Size	Description
Byte 1	u8	1 byte	**Feed Response Code** (see Section 7.4)
Bytes 2-3	i16 (LE)	2 bytes	**Message Length** of the entire payload packet
Byte 4	u8	1 byte	**Exchange Segment** enum (see Section 7.1)
Bytes 5-8	i32 (LE)	4 bytes	**Security ID**

Rust struct:

```rust
#[repr(C, packed)]
struct ResponseHeader {
    feed_response_code: u8,    // 1 byte
    message_length: i16,       // 2 bytes, Little Endian
    exchange_segment: u8,      // 1 byte
    security_id: i32,          // 4 bytes, Little Endian
}
// Total: 8 bytes
```

5.4.2 Ticker Packet (Response Code 2) — Total 16 bytes

Byte Position	Type	Size	Description
Bytes 0-7 (inclusive)	Header	8 bytes	Response Header with code 2
Bytes 8-11	f32 (LE)	4 bytes	**Last Traded Price (LTP)**
Bytes 12-15	i32 (LE)	4 bytes	**Last Trade Time (LTT)** — EPOCH seconds

Rust struct:

```rust
#[repr(C, packed)]
struct TickerPacket {
    header: ResponseHeader,    // 8 bytes
    ltp: f32,                  // 4 bytes, Little Endian
    ltt: i32,                  // 4 bytes, EPOCH seconds, Little Endian
}
// Total: 16 bytes
```

5.4.3 Prev Close Packet (Response Code `6`) — Total 16 bytes

Sent automatically when any instrument is subscribed. Contains previous day data for day-on-day comparison.

Byte Position	Type	Size	Description
Bytes `0-7`	Header	8 bytes	Response Header with code `6`
Bytes `8-11`	`f32` (LE)	4 bytes	**Previous Day Closing Price**
Bytes `12-15`	`i32` (LE)	4 bytes	**Previous Day Open Interest**

Rust struct:

```rust
#[repr(C, packed)]
struct PrevClosePacket {
    header: ResponseHeader,    // 8 bytes
    prev_close: f32,           // 4 bytes
    prev_oi: i32,              // 4 bytes
}
// Total: 16 bytes
```

IMPLEMENTATION NOTE: Store `prev_close` in the in-memory `HashMap<SecurityId, PriceState>` upon receipt. This is needed for `change_pct` computation in the Top Gainers/Losers scanner.

5.4.4 Quote Packet (Response Code `4`) — Total 50 bytes

Byte Position	Type	Size	Description
Bytes 0-7	Header	8 bytes	Response Header with code 4
Bytes 8-11	f32 (LE)	4 bytes	**Last Traded Price (LTP)**
Bytes 12-13	i16 (LE)	2 bytes	**Last Traded Quantity (LTQ)**
Bytes 14-17	i32 (LE)	4 bytes	**Last Trade Time (LTT)** — EPOCH
Bytes 18-21	f32 (LE)	4 bytes	**Average Trade Price (ATP)**
Bytes 22-25	i32 (LE)	4 bytes	**Volume**
Bytes 26-29	i32 (LE)	4 bytes	**Total Sell Quantity**
Bytes 30-33	i32 (LE)	4 bytes	**Total Buy Quantity**
Bytes 34-37	f32 (LE)	4 bytes	**Day Open**
Bytes 38-41	f32 (LE)	4 bytes	**Day Close** (only sent post market close)
Bytes 42-45	f32 (LE)	4 bytes	**Day High**
Bytes 46-49	f32 (LE)	4 bytes	**Day Low**

Rust struct:

```rust
#[repr(C, packed)]
struct QuotePacket {
    header: ResponseHeader,     // 8 bytes
    ltp: f32,                   // 4 bytes
    ltq: i16,                   // 2 bytes
    ltt: i32,                   // 4 bytes, EPOCH
    atp: f32,                   // 4 bytes
    volume: i32,                // 4 bytes
    total_sell_qty: i32,        // 4 bytes
    total_buy_qty: i32,         // 4 bytes
    day_open: f32,              // 4 bytes
    day_close: f32,             // 4 bytes (post-market only)
    day_high: f32,              // 4 bytes
    day_low: f32,               // 4 bytes
}
// Total: 50 bytes
```

5.4.5 OI Packet (Response Code 5) — Total 12 bytes

Sent automatically with Quote subscription. Contains Open Interest for derivatives.

Byte Position	Type	Size	Description
Bytes 0-7	Header	8 bytes	Response Header with code 5
Bytes 8-11	i32 (LE)	4 bytes	**Open Interest** of the contract

Rust struct:

```rust
#[repr(C, packed)]
struct OIPacket {
    header: ResponseHeader,     // 8 bytes
    open_interest: i32,         // 4 bytes
}
// Total: 12 bytes
```

5.4.6 Full Packet (Response Code 8) — Total 162 bytes

This is our primary data packet. Contains everything: LTP, Quote, OI, and 5-level depth.

Byte Position	Type	Size	Description
Bytes 0-7	Header	8 bytes	Response Header with code 8
Bytes 8-11	f32 (LE)	4 bytes	**LTP**
Bytes 12-13	i16 (LE)	2 bytes	**Last Traded Quantity**
Bytes 14-17	i32 (LE)	4 bytes	**Last Trade Time (LTT)** — EPOCH
Bytes 18-21	f32 (LE)	4 bytes	**Average Trade Price**
Bytes 22-25	i32 (LE)	4 bytes	**Volume**
Bytes 26-29	i32 (LE)	4 bytes	**Total Sell Quantity**
Bytes 30-33	i32 (LE)	4 bytes	**Total Buy Quantity**
Bytes 34-37	i32 (LE)	4 bytes	**Open Interest** (derivatives)
Bytes 38-41	i32 (LE)	4 bytes	**Highest OI for the day** (NSE_FNO only)
Bytes 42-45	i32 (LE)	4 bytes	**Lowest OI for the day** (NSE_FNO only)
Bytes 46-49	f32 (LE)	4 bytes	**Day Open**
Bytes 50-53	f32 (LE)	4 bytes	**Day Close** (post-market only)
Bytes 54-57	f32 (LE)	4 bytes	**Day High**
Bytes 58-61	f32 (LE)	4 bytes	**Day Low**
Bytes 62-161	Depth[5]	100 bytes	**5-level Market Depth** (5 × 20 bytes each)

5-Level Depth — Each Level (20 bytes):

Byte Offset	Type	Size	Description
0-3	i32 (LE)	4 bytes	**Bid Quantity**
4-7	i32 (LE)	4 bytes	**Ask Quantity**
8-9	i16 (LE)	2 bytes	**No. of Bid Orders**
10-11	i16 (LE)	2 bytes	**No. of Ask Orders**
12-15	f32 (LE)	4 bytes	**Bid Price**
16-19	f32 (LE)	4 bytes	**Ask Price**

Rust structs:

```rust
#[repr(C, packed)]
struct DepthLevel5 {
    bid_qty: i32,          // 4 bytes
    ask_qty: i32,          // 4 bytes
    bid_orders: i16,       // 2 bytes
    ask_orders: i16,       // 2 bytes
    bid_price: f32,        // 4 bytes
    ask_price: f32,        // 4 bytes
}
// Total: 20 bytes per level

#[repr(C, packed)]
struct FullPacket {
    header: ResponseHeader,    // 8 bytes
    ltp: f32,                  // 4 bytes
    ltq: i16,                  // 2 bytes
    ltt: i32,                  // 4 bytes
    atp: f32,                  // 4 bytes
    volume: i32,               // 4 bytes
    total_sell_qty: i32,       // 4 bytes
    total_buy_qty: i32,        // 4 bytes
    oi: i32,                   // 4 bytes
    highest_oi: i32,           // 4 bytes (NSE_FNO only)
    lowest_oi: i32,            // 4 bytes (NSE_FNO only)
    day_open: f32,             // 4 bytes
    day_close: f32,            // 4 bytes
    day_high: f32,             // 4 bytes
    day_low: f32,              // 4 bytes
    depth: [DepthLevel5; 5],   // 100 bytes (5 × 20)
}
// Total: 162 bytes
```

5.4.7 Index Packet (Response Code 1)

Received for index instruments (NIFTY, BANKNIFTY, etc.). Parse like Ticker but route to index-specific handling.

5.4.8 Market Status Packet (Response Code 7)

Market open/close status notifications. Use to trigger warmup completion and EOD processing.

5.5 Feed Disconnect

To disconnect (client-initiated):

```json
{
  "RequestCode": 12
}
```

Server-initiated disconnect packet:

Byte Position	Type	Size	Description
Bytes 0-7	Header	8 bytes	Response Header with code 50
Bytes 8-9	i16 (LE)	2 bytes	**Disconnection reason code** (see Section 7.6)

Rust implementation must:

1. Detect disconnect packet (response code 50)
2. Read reason code
3. Log reason
4. Auto-reconnect with exponential backoff (unless code 805 = too many connections)

5.6 Binary Parser Implementation Strategy

```rust
```

```rust
fn parse_market_feed(data: &[u8]) -> Result<MarketEvent, ParseError> {
    if data.len() < 8 {
        return Err(ParseError::InsufficientData);
    }

    let response_code = data[0];
    let _msg_length = i16::from_le_bytes([data[1], data[2]]);
    let exchange_segment = data[3];
    let security_id = i32::from_le_bytes([data[4], data[5], data[6], data[7]]);

    match response_code {
        1 => parse_index_packet(data, exchange_segment, security_id),
        2 => parse_ticker_packet(data, exchange_segment, security_id),
        4 => parse_quote_packet(data, exchange_segment, security_id),
        5 => parse_oi_packet(data, exchange_segment, security_id),
        6 => parse_prev_close_packet(data, exchange_segment, security_id),
        7 => parse_market_status_packet(data, exchange_segment, security_id),
        8 => parse_full_packet(data, exchange_segment, security_id),
        50 => parse_disconnect_packet(data),
        _ => Err(ParseError::UnknownResponseCode(response_code)),
    }
}
```

---

6. DHAN FULL MARKET DEPTH — 20-LEVEL & 200-LEVEL PRECISE SPECIFICATION

> **Source:** https://dhanhq.co/docs/v2/full-market-depth/

IMPORTANT: Full Market Depth uses **separate WebSocket connections** from the Live Market Feed. Only **NSE Equity and Derivatives** segments are supported.

6.1 20-Level Market Depth

Connection

Endpoint:

```
wss://depth-api-feed.dhan.co/twentydepth?token={ACCESS_TOKEN}&clientId={CLIENT_ID}&authType=2
```

Limits:

- Up to **50 instruments** per connection
- Can send all 50 instruments in a **single JSON message**
- Can also send multiple messages over a single connection

Subscription

```
{
  "RequestCode": 23,
  "InstrumentCount": 1,
  "InstrumentList": [
    {
      "ExchangeSegment": "NSE_EQ",
      "SecurityId": "1333"
    }
  ]
}
```

Response Header (12 bytes — DIFFERENT from Live Feed!)

Byte Position	Type	Size	Description
Bytes 1-2	i16 (LE)	2 bytes	Message Length
Byte 3	u8	1 byte	Feed Response Code: 41 = Bid, 51 = Ask
Byte 4	u8	1 byte	Exchange Segment
Bytes 5-8	i32 (LE)	4 bytes	Security ID
Bytes 9-12	u32 (LE)	4 bytes	Message Sequence (ignore)

CRITICAL DIFFERENCE: Depth header is **12 bytes**, Live Feed header is **8 bytes**. Response codes are different too: 41 for Bid, 51 for Ask.

Depth Packet Structure (332 bytes total per side)

Byte Position	Type	Size	Description
Bytes 0-11	Header	12 bytes	Response Header (41 for Bid, 51 for Ask)
Bytes 12-331	Depth[20]	320 bytes	20 packets × 16 bytes each

Each Depth Level (16 bytes):

Byte Offset	Type	Size	Description
0-7	f64 (LE)	8 bytes	**Price**
8-11	u32 (LE)	4 bytes	**Quantity**
12-15	u32 (LE)	4 bytes	**No. of Orders**

CRITICAL: 20-level depth uses **f64** **(8 bytes)** for price, while 5-level depth in Full Packet uses **f32** **(4 bytes)**. Different precision!

Rust structs:

```rust
#[repr(C, packed)]
struct DepthResponseHeader {
    message_length: i16,       // 2 bytes
    feed_response_code: u8,    // 1 byte: 41=Bid, 51=Ask
    exchange_segment: u8,      // 1 byte
    security_id: i32,          // 4 bytes
    message_sequence: u32,     // 4 bytes (ignore)
}
// Total: 12 bytes

#[repr(C, packed)]
struct DepthLevel {
    price: f64,              // 8 bytes (double precision!)
    quantity: u32,           // 4 bytes
    num_orders: u32,         // 4 bytes
}
// Total: 16 bytes per level
```

STACKING NOTE: When multiple instruments are subscribed, packets are stacked sequentially: Instrument1-Bid, Instrument1-Ask, Instrument2-Bid, Instrument2-Ask, etc. Parse by reading message_length to determine boundaries.

6.2 200-Level Market Depth

Connection

Endpoint:

```
wss://full-depth-api.dhan.co/twohundreddepth?token={ACCESS_TOKEN}&clientId={CLIENT_ID}&authType=2
```

Limits:

- **ONLY 1 instrument per connection**
- Need separate connection for each instrument
- Use for key instruments: NIFTY, BANKNIFTY, etc.

Subscription (DIFFERENT format from 20-level!)

```json
{
  "RequestCode": 23,
  "ExchangeSegment": "NSE_EQ",
  "SecurityId": "1333"
}
```

NOTE: No `InstrumentList` array — fields are at the top level directly.

Response Header (12 bytes)

Byte Position	Type	Size	Description
Bytes `1-2`	`i16` (LE)	2 bytes	Message Length
Byte `3`	`u8`	1 byte	Feed Response Code: `41` = Bid, `51` = Ask
Byte `4`	`u8`	1 byte	Exchange Segment
Bytes `5-8`	`i32` (LE)	4 bytes	Security ID
Bytes `9-12`	`u32` (LE)	4 bytes	**No. of Rows** (actual depth levels in this response)

CRITICAL: Byte `9-12` is **No. of Rows** (not message sequence like 20-level). Use this to know how many depth levels to parse.

Depth Packet Structure (up to 3212 bytes per side)

Byte Position	Type	Size	Description
Bytes `0-11`	Header	12 bytes	Response Header
Bytes `12-3211`	Depth[200]	3200 bytes max	Up to 200 packets × 16 bytes each

Each level uses the same 16-byte `DepthLevel` struct as 20-level depth (`f64` price, `u32` qty, `u32` orders).

6.3 Connection Management Plan

Connection Type	Endpoint	Max Instruments/Conn	Connections Needed	Total
Live Feed (ticks)	`api-feed.dhan.co`	5000	1 (for 214 F&O)	1
20-Level Depth	`depth-api-feed.dhan.co/twentydepth`	50	ceil(214/50) = 5	5
200-Level Depth	`full-depth-api.dhan.co/twohundreddepth`	1	2-5 (key instruments)	2-5
TOTAL				**8-11**

Ping-Pong for Depth connections: Same as Live Feed — server pings every 10s, auto-pong required, 40s timeout.

6.4 Feed Disconnect (Depth)

Same as Live Feed:

```
{ "RequestCode": 12 }
```

Server disconnect packet has response code `50` with 2-byte reason code.

---

7. DHAN ANNEXURE — EXCHANGE SEGMENTS, INSTRUMENTS, FEED CODES, ERROR CODES

> **Source:** https://dhanhq.co/docs/v2/annexure/

7.1 Exchange Segment Enum (CRITICAL for binary parsing)

Attribute	Exchange	Segment	Binary Enum Value
`IDX_I`	Index	Index Value	`0`
`NSE_EQ`	NSE	Equity Cash	`1`
`NSE_FNO`	NSE	Futures & Options	`2`
`NSE_CURRENCY`	NSE	Currency	`3`
`BSE_EQ`	BSE	Equity Cash	`4`
`MCX_COMM`	MCX	Commodity	`5`

Attribute	Exchange	Segment	Binary Enum Value
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

NOTE: There is **no enum** 6. Values jump from 5 to 7.

Rust enum:

```rust
```

```rust
#[repr(u8)]
enum ExchangeSegment {
    IdxI = 0,
    NseEq = 1,
    NseFno = 2,
    NseCurrency = 3,
    BseEq = 4,
    McxComm = 5,
    // 6 is unused
    BseCurrency = 7,
    BseFno = 8,
}

impl ExchangeSegment {
    fn from_u8(val: u8) -> Option<Self> {
        match val {
            0 => Some(Self::IdxI),
            1 => Some(Self::NseEq),
            2 => Some(Self::NseFno),
            3 => Some(Self::NseCurrency),
            4 => Some(Self::BseEq),
            5 => Some(Self::McxComm),
            7 => Some(Self::BseCurrency),
            8 => Some(Self::BseFno),
            _ => None,
        }
    }

    fn to_string_code(&self) -> &'static str {
        match self {
            Self::IdxI => "IDX_I",
            Self::NseEq => "NSE_EQ",
            Self::NseFno => "NSE_FNO",
            Self::NseCurrency => "NSE_CURRENCY",
            Self::BseEq => "BSE_EQ",
            Self::McxComm => "MCX_COMM",
            Self::BseCurrency => "BSE_CURRENCY",
            Self::BseFno => "BSE_FNO",
        }
    }
}
```

7.2 Instrument Types

Attribute	Description
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock
FUTCOM	Futures of Commodity
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

7.3 Feed Request Codes (Client → Server, JSON)

Code	Action
11	Connect Feed
12	Disconnect Feed
15	Subscribe — Ticker Packet
16	Unsubscribe — Ticker Packet
17	Subscribe — Quote Packet
18	Unsubscribe — Quote Packet
21	Subscribe — Full Packet
22	Unsubscribe — Full Packet
23	Subscribe — Full Market Depth (20-level / 200-level)
24	Unsubscribe — Full Market Depth

7.4 Feed Response Codes (Server → Client, Binary)

Code	Packet Type	Description
1	Index Packet	Index instrument data
2	Ticker Packet	LTP + LTT only
4	Quote Packet	Full quote with OHLC + volume
5	OI Packet	Open Interest (sent with Quote subscription)
6	Prev Close Packet	Previous day close + OI (sent on subscription)
7	Market Status Packet	Market open/close notifications
8	Full Packet	Complete: LTP + Quote + OI + 5-level Depth
41	20/200-Level Bid Depth	Bid side depth data (separate WS)
51	20/200-Level Ask Depth	Ask side depth data (separate WS)
50	Feed Disconnect	Server-initiated disconnection

7.5 Product Types

Attribute	Description
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures & Options
MARGIN	Carry Forward in Futures & Options
CO	Cover Order
BO	Bracket Order

7.6 Order Status Values

Status	Description
TRANSIT	Did not reach exchange server
PENDING	Awaiting execution
CLOSED	Used for Super Order (entry + exit placed)

Status	Description
TRIGGERED	Super Order Target or Stop Loss leg triggered
REJECTED	Rejected by broker/exchange
CANCELLED	Cancelled by user
PART_TRADED	Partial quantity traded
TRADED	Executed successfully

7.7 Trading API Errors

Type	Code	Description
Invalid Authentication	DH-901	Client ID or access token invalid/expired
Invalid Access	DH-902	Not subscribed to Data/Trading APIs
User Account	DH-903	Account issues (segments not activated)
Rate Limit	DH-904	Too many requests, throttle API calls
Input Exception	DH-905	Missing required fields, bad parameter values
Order Error	DH-906	Incorrect order request
Data Error	DH-907	Unable to fetch data (wrong params / no data)
Internal Server Error	DH-908	Server processing failure (rare)
Network Error	DH-909	API unable to communicate with backend
Others	DH-910	Other errors

7.8 Data API / WebSocket Errors

Code	Description
800	Internal Server Error
804	Requested instruments exceed limit
805	Too many requests/connections (may be blocked)
806	Data APIs not subscribed

Code	Description
807	Access token expired
808	Authentication Failed (Client ID or Token invalid)
809	Access token invalid
810	Client ID invalid
811	Invalid Expiry Date
812	Invalid Date Format
813	Invalid SecurityId
814	Invalid Request

8. DHAN AUTHENTICATION & TOKEN MANAGEMENT

8.1 Access Token Generation (via API + TOTP)

Endpoint:

```
POST https://auth.dhan.co/app/generateAccessToken?dhanClientId={CLIENT_ID}&pin={PIN}&totp={TOTP_CODE}
```

Response:

```json
{
  "dhanClientId": "1000000401",
  "dhanClientName": "JOHN DOE",
  "dhanClientUcc": "ABCD12345E",
  "givenPowerOfAttorney": false,
  "accessToken": "eyJ...",
  "expiryTime": "2026-01-01T00:00:00.000"
}
```

Token Validity: 24 hours from generation. Must regenerate daily.

8.2 Token Renewal

Endpoint:

```
POST https://api.dhan.co/v2/RenewToken
```

Headers:

```
access-token: {CURRENT_JWT_TOKEN}
dhanClientId: {CLIENT_ID}
```

NOTE: Only works for **active** tokens. Expired tokens cannot be renewed — must generate a new one.

8.3 TOTP Configuration

- TOTP follows RFC 6238 (Time-based One-Time Password)
- Generates 6-digit code from shared secret every 30 seconds
- Use Rust crate `totp-rs` 5.x for server-side TOTP generation
- TOTP secret is stored in AWS Secrets Manager (never hardcoded)

8.4 Token Lifecycle Automation

```
┌──────────────────────────────────────────────────────────────────────┐
│ DAILY TOKEN LIFECYCLE (Automated)                                    │
│                                                                      │
│ 08:30 IST: Generate TOTP code (totp-rs)                              │
│ 08:30 IST: POST generateAccessToken                                  │
│ 08:30 IST: Store token in Redis + in-memory                          │
│ 09:00 IST: Market opens, WS connections use token                    │
│ 15:30 IST: Market closes                                             │
│ 20:00 IST: POST RenewToken (refresh for next 24hr)                   │
│ Next day 08:30 IST: Regenerate fresh token                           │
└──────────────────────────────────────────────────────────────────────┘
```

8.5 Static IP Management (SEBI Compliance)

Order placement, modification, and cancellation APIs REQUIRE Static IP whitelisting.

Set IP:

```
POST https://api.dhan.co/v2/ip/setIP
Headers: access-token: {TOKEN}
Body: {
  "dhanClientId": "1000000001",
  "ip": "10.200.10.10",
  "ipFlag": "PRIMARY"     // or "SECONDARY"
}
```

Get IP:

```
GET https://api.dhan.co/v2/ip/getIP
Headers: access-token: {TOKEN}
```

Response:

```
json

{
  "modifyDateSecondary": "2025-09-30",
  "secondaryIP": "10.420.43.12",
  "modifyDatePrimary": "2025-09-28",
  "primaryIP": "10.420.29.14"
}
```

Rules:

- Once IP is set, cannot modify for **7 days**
- Supports both IPv4 and IPv6
- PRIMARY and SECONDARY IPs supported
- Required for: Normal Orders, Super Orders, Forever Orders

---

9. DHAN ORDER MANAGEMENT API — PRECISE SPECIFICATION

Base URL: `https://api.dhan.co/v2` **Auth Header:** `access-token: {JWT_TOKEN}` **Content-Type:** `application/json`

9.1 Place Order

```
POST https://api.dhan.co/v2/orders
```

Request Body:

```
json
```

```
{
  "dhanClientId": "1000000003",
  "correlationId": "123abc678",
  "transactionType": "BUY",
  "exchangeSegment": "NSE_EQ",
  "productType": "INTRADAY",
  "orderType": "MARKET",
  "validity": "DAY",
  "securityId": "11536",
  "quantity": "5",
  "disclosedQuantity": "",
  "price": "",
  "triggerPrice": "",
  "afterMarketOrder": false,
  "amoTime": "",
  "boProfitValue": "",
  "boStopLossValue": ""
}
```

Response:

```
json
{
  "orderId": "112111182198",
  "orderStatus": "PENDING"
}
```

9.2 Modify Order

```
PUT https://api.dhan.co/v2/orders/{order-id}
```

Request Body:

```
json
```

```
{
  "dhanClientId": "1000000009",
  "orderId": "112111182045",
  "orderType": "LIMIT",
  "legName": "",
  "quantity": "40",
  "price": "3345.8",
  "disclosedQuantity": "10",
  "triggerPrice": "",
  "validity": "DAY"
}
```

9.3 Cancel Order

```
DELETE https://api.dhan.co/v2/orders/{order-id}
```

9.4 Get Order by ID

```
GET https://api.dhan.co/v2/orders/{order-id}
```

9.5 Get All Orders (Order Book)

```
GET https://api.dhan.co/v2/orders
```

9.6 Get All Trades (Trade Book)

```
GET https://api.dhan.co/v2/trades
```

9.7 Slice Order (F&O freeze qty)

```
POST https://api.dhan.co/v2/orders/slicing
```

Same body as Place Order. Automatically splits into multiple orders for freeze-limit quantities.

9.8 Order Parameters Reference

Field	Type	Required	Values
dhanClientId	string	YES	Your Dhan Client ID
correlationId	string	NO	Your internal reference ID
transactionType	string	YES	BUY, SELL

Field	Type	Required	Values
exchangeSegment	string	YES	NSE_EQ, NSE_FNO, BSE_EQ, MCX_COMM, etc.
productType	string	YES	CNC, INTRADAY, MARGIN, CO, BO
orderType	string	YES	MARKET, LIMIT, SL, SLM
validity	string	YES	DAY, IOC
securityId	string	YES	Exchange Security ID
quantity	string	YES	Order quantity
price	string	Conditional	Required for LIMIT/SL
triggerPrice	string	Conditional	Required for SL/SLM
disclosedQuantity	string	NO	Disclosed quantity
afterMarketOrder	bool	NO	true for AMO
amoTime	string	NO	PRE_OPEN, OPEN, OPEN_30, OPEN_60

9.9 Rate Limits

- Order APIs: **10 orders per second**
- Data APIs: No rate limit on minute/hourly timeframes
- Up to 100,000 requests per day for data

---

10. DHAN LIVE ORDER UPDATE WEBSOCKET

Endpoint:

```
wss://api-order-update.dhan.co
```

Auth message (sent after connection):

```
json
```

```
{
  "LoginReq": {
    "MsgCode": 42,
    "ClientId": "{CLIENT_ID}",
    "Token": "{ACCESS_TOKEN}"
  }
}
```

Order update response (JSON, not binary):

```json
{
  "Data": {
    "Exchange": "NSE",
    "Segment": "E",
    "SecurityId": "14366",
    "ClientId": "1000000001",
    "ExchOrderNo": "1400000000404591",
    "OrderNo": "1124091136546",
    "Product": "C",
    "TxnType": "B",
    "OrderType": "LMT",
    "Validity": "DAY",
    "RemainingQuantity": 1,
    "Quantity": 1,
    "TradedQty": 0,
    "Price": 13,
    "TriggerPrice": 0,
    "TradedPrice": 0,
    "AvgTradedPrice": 0,
    "Status": "Cancelled",
    "CorrelationId": "123abc678",
    "Remarks": "NR"
  }
}
```

---

11. O(1) COMPONENT SCORING

Component	Score	O(1)?	Notes
Rust Language	10/10	YES	No GC, zero-cost abstractions
Tokio Async Runtime	10/10	YES	Work-stealing scheduler

Component	Score	O(1)?	Notes
tokio-tungstenite	10/10	YES	Zero-copy frame parsing
Manual byte slicing (binary parser)	10/10	YES	Fixed-size packets, O(1) guaranteed
Redis GET/SET	10/10	YES	In-memory hash, <1μs
QuestDB Write (ILP)	10/10	YES	Append-only columnar
QuestDB Read (Mat View)	10/10	YES	Pre-computed, <2ms
QuestDB Arrays	10/10	YES	N-dimensional for order books
Crossbeam Lock-free	10/10	YES	Lock-free concurrent structures
DashMap	10/10	YES	Concurrent HashMap
RustRover 2025.2	10/10	YES (dev)	Not in hot path
Prometheus (sampled)	9/10	PARTIAL	Record O(1), scrape O(n) async
Grafana	9/10	NO (read-only)	Queries materialized views

12. COMPLETE RUST CRATE DEPENDENCY MAP

Purpose	Crate	Version	O(1)?
Async Runtime	tokio	1.x (full features)	YES
WebSocket Client	tokio-tungstenite	0.21.x	YES
DB Ingestion (QuestDB)	questdb-rs	4.x (with ndarray)	YES
DB Query (QuestDB PGWire)	tokio-postgres	0.7.x (with chrono)	YES
DB Connection Pool	deadpool-postgres	latest	YES
Redis Client	redis (tokio feature)	0.25.x	YES
Lock-free Structures	crossbeam	0.8.x	YES
Concurrent HashMap	dashmap	5.x	YES
Ordered Float	ordered-float	4.x	YES
Serialization	serde + serde_json	1.x	YES

Purpose	Crate	Version	O(1)?
Time/Dates	chrono	0.4.x	YES
Metrics (Prometheus)	prometheus	0.13.x	YES
HTTP Client (REST APIs)	reqwest (tokio)	0.12.x	YES
TOTP Auth	totp-rs	5.x	YES
TLS	rustls or tokio-native-tls	latest	YES
Logging	tracing + tracing-subscriber	0.1.x	YES
AWS Secrets Manager	aws-sdk-secretsmanager	latest	YES
AWS Config	aws-config	latest	YES
N-dimensional Arrays	ndarray	0.15.x	YES
Byte Manipulation	bytes	1.x	YES
UUID Generation	uuid	1.x	YES

13. DOCKER-FIRST STACK

13.1 Local docker-compose

Service	Image	Ports	Purpose
trading-engine	Custom Rust image	8080	Main app (Rust + Tokio)
questdb	questdb/questdb:9.x	9000 (HTTP), 8812 (PGWire), 9009 (ILP)	Time-series DB
redis	redis:7-alpine	6379	Cache & state
prometheus	prom/prometheus:latest	9090	Metrics
grafana	grafana/grafana:latest	3000	Dashboards (QuestDB plugin)
jaeger	jaegertracing/all-in-one:latest	16686	Tracing (dev only)

13.2 AWS ECS Fargate Stack

Service	AWS Equivalent
trading-engine	ECS Fargate Task
questdb	EC2 with EBS or QuestDB Cloud
redis	ElastiCache Redis
prometheus	Amazon Managed Prometheus
grafana	Amazon Managed Grafana

13.3 QuestDB Docker Config

- Data Volume: `/var/lib/questdb`
- HTTP API: 9000
- PGWire: 8812
- ILP: 9009
- Min Heap: `-Xms512m`
- Max Heap: `-Xmx4g`

---

14. LATENCY BUDGET (100K Ticks/Sec)

Stage	Latency	Complexity	Component
WebSocket tick reception	5μs	O(1)	tokio-tungstenite
Binary parsing	10μs	O(1)	Manual byte slicing
Deduplication check	2μs	O(1)	DashMap lookup
Indicator computation (90x)	45μs	O(1) each	Incremental state
Strategy evaluation (90x)	18μs	O(1) each	Pre-computed inputs
Order decision	5μs	O(1)	Rule evaluation
Candle update (21 timeframes)	5μs	O(1) each	Arithmetic ops
Top gainers/losers update	1μs	O(1)	BTreeMap (K=214)
Redis cache update	1μs	O(1)	Redis SET

Stage	Latency	Complexity	Component
QuestDB ILP write (async)	0μs (non-blocking)	O(1)	questdb-rs async
Metrics (1% sampled)	0.01μs	O(1)	Prometheus histogram
TOTAL	**~92μs**	**O(1)**	**UNDER 500μs BUDGET**

15. END-TO-END O(1) PROCESSING PIPELINE

Stage	Complexity	Component	Notes
1. Dhan WS Connect	O(1)	tokio-tungstenite	wss://api-feed.dhan.co?version=2&token=...
2. Live Feed Tick	O(1)	Binary frame receive	Up to 5000 instruments, RequestCode 21
3. Full Market Depth	O(1)	Binary 20/200 level	Separate WS: depth-api-feed.dhan.co, full-depth-api.dhan.co
4. Binary Deser	O(1)	Manual byte slice	LE, fixed-size: 8-byte header, route by response code
5. Tick Dedup	O(1)	DashMap<(security_id, seq)>	Composite key lookup
6. Price State Update	O(1)	HashMap update	LTP, volume, OI, change%
7. Prev Close Store	O(1)	HashMap insert	From response code 6 packet on subscription
8. Top Gainers/Losers	O(1)	BTreeMap (K=214)	Bounded sorted structure
9. Candle Update (21)	O(1) each	In-memory OHLCV	Arithmetic: max, min, sum, assign
10. Indicator Compute	O(1) each	Incremental state	EMA, RSI, MACD, Bollinger, etc.
11. Strategy Evaluate	O(1) each	Rule engine	Consume pre-computed indicators
12. Order Decision	O(1)	Signal gen	Buy/Sell/Hold
13. Order Placement	O(1)	POST https://api.dhan.co/v2/orders	Async, non-blocking

Stage	Complexity	Component	Notes
14. Order Update	O(1)	WS JSON from Dhan	wss://api-order-update.dhan.co
15. QuestDB Write	O(1)	questdb-rs ILP batch	Non-blocking buffer flush
16. Redis Cache	O(1)	Redis SET	Latest state
17. Metrics	O(1)	Prometheus histogram	1% sampling
18. Dashboard Query	O(1)	QuestDB mat view	<2ms response

16. WARMUP DATA & HISTORICAL DATA

16.1 Warmup Window

- Minimum: 14 days historical data
- Maximum: 1 month
- Source: Dhan Historical Data API (intraday minute data, last 5 trading days) + accumulated real-time
- All 21 candle timeframes warmed up before live trading begins

16.2 QuestDB Data Retention (TTL)

Data Type	TTL
Raw ticks	7 DAYS
5s-60s candles	30 DAYS
1m-30m candles	90 DAYS
Daily aggregates	365 DAYS
Orders/trades	No TTL (compliance)

17. DELIVERABLES CHECKLIST

17.1 Rust Application Crate Structure

```
algotrading-engine/
├── src/
```

```
│   ├── main.rs              -- Entry point, Tokio runtime bootstrap
│   ├── config/              -- Constants: AppConstants, DbConstants, ApiConstants
│   │   └── dhan_constants.rs   -- All Dhan enums, URLs, request/response codes
│   ├── auth/                -- TOTP generation, token management, AWS Secrets
│   │   ├── totp.rs          -- TOTP code generation (totp-rs)
│   │   ├── token.rs         -- generateAccessToken + RenewToken
│   │   └── secrets.rs       -- AWS Secrets Manager integration
│   ├── ws/                  -- WebSocket clients
│   │   ├── live_feed.rs     -- Live Market Feed (api-feed.dhan.co)
│   │   ├── depth_20.rs      -- 20-Level Depth (depth-api-feed.dhan.co)
│   │   ├── depth_200.rs     -- 200-Level Depth (full-depth-api.dhan.co)
│   │   ├── order_update.rs  -- Live Order Update WS
│   │   └── connection_mgr.rs   -- Connection pool, reconnection logic, ping-pong
│   ├── parser/              -- Binary packet parsers
│   │   ├── header.rs        -- 8-byte live feed header + 12-byte depth header
│   │   ├── ticker.rs        -- Response code 2: Ticker (16 bytes)
│   │   ├── quote.rs         -- Response code 4: Quote (50 bytes)
│   │   ├── oi.rs            -- Response code 5: OI (12 bytes)
│   │   ├── prev_close.rs    -- Response code 6: Prev Close (16 bytes)
│   │   ├── full_packet.rs   -- Response code 8: Full (162 bytes)
│   │   ├── depth.rs         -- Response codes 41/51: 20/200-level depth
│   │   ├── disconnect.rs    -- Response code 50: Disconnect
│   │   └── router.rs        -- Route by response code to correct parser
│   ├── dedup/               -- Deduplication engine (DashMap-based)
│   ├── candles/             -- Candle generator (21 timeframes, O(1) each)
│   ├── indicators/          -- Indicator engine (100+ indicators, O(1) each)
│   ├── strategies/          -- Strategy engine (100+ strategies, O(1) each)
│   ├── orders/              -- Order management
│   │   ├── placer.rs        -- POST /v2/orders, /v2/orders/slicing
│   │   ├── modifier.rs      -- PUT /v2/orders/{id}
│   │   ├── canceller.rs     -- DELETE /v2/orders/{id}
│   │   └── book.rs          -- GET /v2/orders, /v2/trades
│   ├── db/                  -- QuestDB client
│   │   ├── writer.rs        -- questdb-rs ILP ingestion (ticks, depth, orders)
│   │   ├── reader.rs        -- tokio-postgres queries (mat views, candles)
│   │   └── schema.rs        -- Schema initialization + mat view creation
│   ├── cache/               -- Redis client (state caching)
│   ├── scanner/             -- Top gainers/losers/volume/OI scanner
│   ├── metrics/             -- Prometheus metrics (sampled recording)
│   ├── depth/               -- Full market depth handler (20 + 200 level)
│   └── ip/                  -- Static IP management (SEBI compliance)
├── Cargo.toml               -- All crate dependencies
├── Dockerfile               -- Multi-stage build for minimal image
└── docker-compose.yml       -- Full local stack with QuestDB
```

17.2 QuestDB Schema Files

```
schema/
├── 001_create_ticks.sql
├── 002_create_depth_20.sql
├── 003_create_depth_200.sql
├── 004_create_orders.sql
├── 005_create_candle_views.sql     -- All 21 materialized views
├── 006_create_scanner_views.sql    -- latest_prices view
└── 007_create_indicator_state.sql
```

17.3 Documentation

```
docs/
├── ARCHITECTURE.md
├── DHAN_API_MAPPING.md             -- This section of the document
├── BINARY_PROTOCOL.md              -- Byte-level packet specs
├── DATABASE_SCHEMA.md
├── INDICATORS.md
├── STRATEGIES.md
├── DEPLOYMENT.md
├── RUNBOOK.md
├── LATENCY_BUDGET.md
└── DEPTH_INTEGRATION.md
```

17.4 Infrastructure

```
infra/
├── Dockerfile
├── docker-compose.yml
├── ecs-task-definition.json
├── grafana/
├── prometheus/
└── questdb/
```

---

18. FINAL SUMMARY

Characteristic	Status	Guarantee
Full Automation	YES	Zero manual steps
O(1) Performance (ALL stages)	YES	<500μs p99 end-to-end

Characteristic	Status	Guarantee
O(1) Database Reads	YES (QuestDB)	<2ms via materialized views
O(1) Database Writes	YES (QuestDB ILP)	Async append, millions rows/sec
100% Tick Capture	YES	Never miss a tick
100K TPS Capacity	YES	Handles extreme load
21 Candle Timeframes	YES	5s to 30m, all O(1)
Top Gainers/Losers	YES	All 214 F&O, real-time
Full Market Depth	YES	20-level + 200-level
5-Level Depth in Full Packet	YES	From RequestCode 21
100+ Indicators	YES	Scalable, O(1) each
100+ Strategies	YES	Parallel execution, O(1) each
Precise Dhan Binary Protocol	YES	Byte-level specification
All Exchange Segments Mapped	YES	Enum 0-8 (no 6)
All Feed Request/Response Codes	YES	11,12,15-18,21-24 / 1,2,4,5,6,7,8,41,51,50
All Error Codes Mapped	YES	DH-901 to DH-910, 800-814
Authentication Automated	YES	TOTP + 24hr token + renewal
Static IP Managed	YES	Set/Get/Modify API
Order API Complete	YES	Place/Modify/Cancel/Slice/Book
Live Order Updates	YES	Separate WS, JSON format
Local-AWS Parity	YES	Identical execution
Docker-Native	YES	Runs everywhere
SEBI Compliant	YES	Static IP whitelisting
Nanosecond Timestamps	YES (QuestDB)	TIMESTAMP_NS
Little Endian Confirmed	YES	All Dhan binary data

DOCUMENT STATUS: COMPLETE | V3 WITH PRECISE DHAN API MAPPINGS | READY FOR CLAUDE CODE IMPLEMENTATION

Provide this entire document to Claude Code for full implementation.